Write the kernel in (1) C program; (2) an x86-64 assembly language; (3) x86-64 SIMD AVX2 assembly language using XMM register; (4) x86-64 SIMD AVX2 assembly language using YMM register. The kernel is to perform **Euclidean Norms** on vector *A*, and placing the result in scalar *Z*.

Input: Scalar variable *n* (integer) contains the length of the vector; Vector *A* are double-precision floats.

Process: $\mathrm{Z} = \sqrt{\sum_{i=0}^{n-1} A_i^2} \quad \text{for } i = 0, 1, \ldots, n-1$

Output: store the result in double precision scalar variable *Z*. Display the result for verification across all 4 kernels.

AHAness: Is this memory-bound or compute-bound? Will SIMD YMM be faster than optimized C?

1.) Write a C main program to call the kernels of the C version, non-SIMD x86-64 assembly language, SIMD x86 assembly language using the XMM register, and SIMD x86 assembly language using the YMM register.

2.) Time the kernel portion only. Do this for both DEBUG and RELEASE versions of Visual Studio

3.) For each kernel version, time the process for vector size $n = \{2^{20}, 2^{26}, \text{and } 2^{30}\}$. If 2^{30} is not possible, reduce it to 2^{28} or 2^{29}, depending on the memory capability of your machine.

3a.) Initialize also the vector size with extra elements to check if your program can handle boundary situations. For example, if a SIMD register can load 4 elements at a time but the vector size is 1003 elements, how will your system handle "boundary" elements?

4.) For each kernel, execute at least 30 times and get the average execution time.

5.) For the data, initialize each vector with values of your choice. Please document this value.

Example:

```
for (int i = 0; i < n; i++) {
    x[i] = sin(i * 0.001);
    y[i] = cos(i * 0.002);
}
```

6.) Check the correctness of your output. Thus, if the C version is your "sanity check answer key," then the output of the x86-64 version and both SIMD versions must be checked against the C version and output accordingly (i.e., the x86-64 kernel output is correct, etc.).

7.) Place the result in GitHub (ensure I can access your GitHub). The GitHub report should be inline and NOT linked to external files.

8.) The GitHub repository contains the following:

a.) Readme section containing the following report:

i.) screenshot of the program output with execution time for all cases

ii.) comparative table of execution time as well as analysis of the performance of different kernels (how many times faster, why is it faster, etc.)

iii.) screenshot of the program output with correctness check (C)

iv.) screenshot of the program output, including correctness check (x86-64)

v.) screenshot of the program output, including correctness check (SIMD XMM register)

vi.) screenshot of the program output, including correctness check (SIMD, YMM register)

vii.) screenshot of boundary check for SIMD XMM and SIMD YMM.

viii.) Analyze the results obtained from all platforms. Which one is faster, why is it faster, why speedup is not comparable with the theorical frameworks, etc.

ix.) Discuss the problems encountered and solutions made, unique methodology used, AHA moments, etc.

b.) All files under the Visual Studio project folder (source code: C program, x86-64 non-SIMD, x86-64 SIMD XMM version, x86-64 SIMD YMM version, and all other required files) to load and execute your program.

Rubric:

C main program with initialization and correct call/ passing parameters to C, x86-64, and SIMD version	5
Display timing + correctness (C version)	10
Display timing + correctness (x86-64)	10
Display timing + correctness (SIMD XMM)	10
Display timing + correctness (SIMD YMM)	10
***with boundary handling	10
Comparative result (table)	5
Analysis of the result	20
Discussion of implementation	10
Completeness of the file uploaded	5
Following instructions	5